

OMNIVAX PLATFORM - COMPLETE SYSTEM DOCUMENTATION

Document Version: 1.0
Analysis Date: Based on codebase snapshot
Confidence Level: HIGH (full codebase access)

1. SYSTEM OVERVIEW

What the System Does (Based on Code Evidence)

Omnivax is a **plant disease intelligence platform** that provides:

- A cloud-based API for running ML inference on plant/leaf images
- A model registry for managing, discovering, and deploying computer vision models
- Real-time prediction capabilities with confidence scoring
- Benchmark tracking for model performance evaluation
- Usage quota management with tiered pricing
- AI-powered agronomic recommendations via Gemini integration

Core Purpose

Democratize access to high-performance computer vision systems for farmers, researchers, and ag-tech startups by providing:

1. Pre-trained models for plant disease detection (corn, tomato)
2. Infrastructure for developers to upload and manage their own models
3. Multi-modal access (Web UI, CLI, REST API)

Key Capabilities

Capability	Implementation Status
Image-based disease classification	✅ Complete
Model registry & discovery	✅ Complete
User authentication (Supabase + local)	✅ Complete (hybrid)
Role-based access control	✅ Complete
Live video inference	✅ Complete
Batch benchmarking	✅ Complete

Capability	Implementation Status
Usage quota enforcement	⚠️ Partial (not enforced in predict flow)
AI chat assistant	✅ Complete
Analytics dashboard	✅ Complete (enterprise only)
CLI tool for developers	✅ Complete

2. FEATURE INVENTORY

Authentication & User Management

Feature	Description	Related Files	Status
Supabase OAuth Auth	Primary authentication via Supabase	api/auth.py, lib/supabase.ts	✅ Complete
Legacy JWT Auth	Fallback local JWT authentication	api/auth.py	✅ Complete
User Registration	Email/password registration	api/auth.py, schemas/user.py	✅ Complete
Auto-provisioning	Creates local DB user from Supabase auth	api/auth.py	✅ Complete
Role-based Access	STANDARD, DEVELOPER, ENTERPRISE roles	db/models.py, api/auth.py	✅ Complete
Session Management	Token-based with localStorage persistence	hooks/useUser.ts	✅ Complete

Model Registry

Feature	Description	Related Files	Status
Model Listing	List all registered models	api/models.py, registry/model_registry.py	✅ Complete
Model Search	Text-based search across metadata	api/models.py, registry/model_registry.py	✅ Complete
Model Filtering	Filter by tags, plant, disease, tier	registry/model_registry.py	✅ Complete

Feature	Description	Related Files	Status
Model Detail View	Full metadata retrieval	api/models.py, registry/model_registry.py	✓ Complete
Model Upload	File upload for model artifacts	api/models.py	✓ Complete
Model Registration	JSON metadata registration	api/models.py, registry/model_registry.py	✓ Complete
Pricing Tiers	Free/Pro/Enterprise tier definitions	models/metadata/pricing_tiers.json	✓ Complete

Prediction & Inference

Feature	Description	Related Files	Status
Single Image Prediction	Upload image, get top-k predictions	api/predict.py, inference/efficientnet_adapter.py	✓ Complete
Live Video Inference	Real-time webcam stream processing	api/predict.py, app/predict/page.tsx	✓ Complete
Prediction History	User-specific prediction logs	api/predict.py, db/models.py	✓ Complete
Adapter Pattern	PyTorch EfficientNet B0 inference	inference/efficientnet_adapter.py	✓ Complete
Model Caching	In-memory adapter caching per model_id	api/predict.py	✓ Complete

Benchmarking

Feature	Description	Related Files	Status
Benchmark Execution	Run accuracy/latency benchmarks	api/benchmarks.py, services/benchmark_service.py	✓ Complete
Manifest-based Testing	Evaluate against labeled datasets	services/benchmark_service.py	✓ Complete
Synthetic Baselines	Demo mode without real data	services/benchmark_service.py	✓ Complete
Benchmark Comparison	Compare multiple models	api/benchmarks.py	✓ Complete

Feature	Description	Related Files	Status
Metrics Tracking	Accuracy, Precision, Recall, F1, Latency, Throughput	schemas/benchmarks.py	 Complete

Billing & Usage

Feature	Description	Related Files	Status
Usage Check API	Check quota availability	api/billing.py, registry/model_registry.py	 Complete
Usage Recording	Track API calls per user/model	api/billing.py, registry/model_registry.py	 Complete
Pricing Tier Display	Show available tiers	api/billing.py	 Complete
Quota Enforcement Logic	Daily quota calculation	registry/model_registry.py	 Not integrated into predict flow

AI Assistant

Feature	Description	Related Files	Status
Gemini Chat Integration	AI agronomist recommendations	api/chat.py	 Complete
Context-aware Responses	Uses prediction results as context	api/chat.py, components/AiChatModal.tsx	 Complete
Conversation History	Maintains chat context	api/chat.py	 Complete

Analytics

Feature	Description	Related Files	Status
Global Summary	Total scans, users, avg confidence	api/analytics.py	 Complete
Detection Trends	Time-series detection frequency	api/analytics.py	 Complete
Model Performance	Per-model usage and confidence stats	api/analytics.py	 Complete
Role Restriction	Enterprise/Developer only	api/analytics.py	 Complete

CLI Tool

Feature	Description	Related Files	Status
Authentication	Login/logout via Supabase	cli/omnivax/auth.py	✅ Complete
Model Listing	View registry from terminal	cli/omnivax/models.py	✅ Complete
Model Info	Detailed model metadata	cli/omnivax/models.py	✅ Complete
Model Push	Upload models via CLI	cli/omnivax/models.py	✅ Complete
Remote Prediction	Run inference from CLI	cli/omnivax/run.py	✅ Complete

3. MODULE / COMPONENT BREAKDOWN

Backend Application (/workspace/backend/app)

Module: main.py

Responsibility: FastAPI application entry point, router registration, database initialization

Key Functions:

- App initialization with CORS middleware
- Database table creation (Base.metadata.create_all)
- Seed data population from registry on startup
- Router inclusion for all API endpoints

Dependencies:

- FastAPI, SQLAlchemy, all API routers, settings, model_service

Design Assessment: ⚠️ Problematic

- Database seeding occurs at import time (before app starts)
 - No migration strategy (creates tables on every start)
 - Hardcoded static file mounting
-

Module: core/settings.py

Responsibility: Centralized configuration management

Key Classes:

python

Design Assessment: ✅ Good

```
class Settings(BaseSettings):
```

```
    app_name: str
```

```
    app_version: str
```

```
    debug: bool
```

```
    default_model_id: str
```

```
    SECRET_KEY: str
```

```
    DATABASE_URL: str
```

```
    GEMINI_API_KEY: str
```

```
    SUPABASE_URL: str
```

```
    SUPABASE_JWT_SECRET: str
```

```
    workspace_root: Path
```

```
    models_metadata_dir: Path
```

```
    models_artifacts_dir: Path
```

```
    benchmark_runs_dir: Path
```

- Uses pydantic-settings for env var loading
- Clear separation of required vs optional settings
- Sensible defaults provided

Module: db/models.py

Responsibility: SQLAlchemy ORM models

Key Models:

1. **User** - Authentication and role management
2. **ModelCatalog** - Model metadata storage
3. **PredictionLog** - Prediction history tracking

Design Assessment:  **Good**

- Proper foreign key relationships
- UUID primary keys for users
- Enum for UserRole

- Timestamps with timezone awareness
-

Module: db/database.py

Responsibility: Database connection management

Key Components:

- SQLAlchemy engine with connection pooling
- Session factory
- Dependency injection for FastAPI

Design Assessment:  **Good**

- Connection pool configured (size=10, max_overflow=20)
 - Pool pre-ping enabled for stale connection handling
 - Proper session cleanup in get_db()
-

Module: registry/model_registry.py

Responsibility: File-based model metadata management

Key Methods:

- load() - Load all JSON metadata files
- list_models(), get_model_summary(), get_model_detail()
- search(), filter_models() - Discovery
- check_access(), record_usage() - Quota management
- add_benchmark(), list_benchmarks() - Benchmark tracking

Data Files Managed:

- models.json - Model index
- tags.json - Available tags
- benchmarks.json - Benchmark results
- pricing_tiers.json - Tier definitions
- usage.json - Per-user usage tracking

Design Assessment:  **Problematic**

- File-based storage not suitable for production

- No concurrency handling for writes
 - Usage tracking separate from main database
 - No atomic operations
-

Module: `inference/efficientnet_adapter.py`

Responsibility: PyTorch model inference wrapper

Key Class:

python

class EfficientNetB0Adapter:

```
def __init__(artifact_path, model_meta)
def _load_model() -> torch.nn.Module
def _build_transform() -> transforms.Compose
def predict_image_bytes(image_bytes, top_k) -> list[dict]
```

Design Assessment: ⚠️ **Limited**

- Only supports EfficientNet B0 architecture
 - Hardcoded transform parameters from metadata
 - No GPU memory management
 - No batch prediction support
-

Module: `services/model_service.py`

Responsibility: Registry initialization

Implementation:

python

```
registry = ModelRegistry(settings.models_metadata_dir)
registry.load()
```

Design Assessment: ⚠️ **Minimal**

- Just a wrapper importing registry
- Could be eliminated or expanded

Module: services/benchmark_service.py

Responsibility: Benchmark execution logic

Key Functions:

- `_load_adapter()` - Load model for benchmarking
- `_run_manifest_benchmark()` - Real dataset evaluation
- `run_benchmark()` - Main entry point (supports synthetic mode)

Metrics Computed:

- Accuracy, Precision (weighted), Recall (weighted), F1 (weighted)
- Latency P50, P95
- Throughput (images/sec)

Design Assessment:  **Good**

- Supports both real and synthetic benchmarks
- Proper metric calculations using sklearn
- Handles edge cases (empty manifests)

Module: api/auth.py

Responsibility: Authentication endpoints

Endpoints:

- `POST /auth/register` - New user registration
- `POST /auth/login` - Legacy JWT login
- `GET /auth/me` - Current user profile

Key Functions:

- `verify_password()`, `get_password_hash()` - bcrypt utilities
- `create_access_token()` - JWT generation
- `get_current_user()` - Dual-mode auth (Supabase + fallback)
- `require_role()` - RBAC decorator factory

Design Assessment:  **Complex**

- Dual authentication paths create confusion

- Password truncation to 71 bytes for bcrypt (correct but unusual)
 - Auto-provisioning logic is complex
 - Legacy JWT should be deprecated
-

Module: api/models.py

Responsibility: Model CRUD operations

Endpoints:

- POST /models/upload - File upload + registration
- POST /models - Metadata-only registration
- GET /models - List all models
- GET /models/search - Text search
- GET /models/{model_id} - Get detail

Design Assessment:  **Good**

- Proper role-based access (DEVELOPER, ENTERPRISE only)
 - Schema validation via Pydantic
 - Helper functions for response transformation
-

Module: api/predict.py

Responsibility: Inference requests

Endpoints:

- POST /predict - Run prediction
- GET /predict/history - User prediction history

Key Features:

- Adapter caching (_ADAPTER_CACHE)
- Image validation (JPEG/PNG only)
- Automatic logging to database
- Authentication required

Design Assessment:  **Issues**

- No quota enforcement before prediction

- Cache never invalidated (stale models)
 - Optional auth comment contradicts implementation
-

Module: api/benchmarks.py

Responsibility: Benchmark management

Endpoints:

- GET /benchmarks - List benchmarks
- POST /benchmarks/run - Execute new benchmark
- GET /benchmarks/compare - Compare models

Design Assessment:  **Good**

- Clean normalization function for responses
 - Proper error handling
 - Flexible comparison endpoint
-

Module: api/billing.py

Responsibility: Usage tracking

Endpoints:

- GET /usage/pricing - List pricing tiers
- POST /usage/check-access - Check quota
- POST /usage/record - Record usage

Design Assessment:  **Disconnected**

- Endpoints exist but not called in predict flow
 - Manual usage recording required
 - Should be automatic
-

Module: api/chat.py

Responsibility: AI assistant

Endpoints:

- POST /chat/ - Chat with Gemini

Key Features:

- System instruction for agronomist persona
- Prediction context injection
- Conversation history support

Design Assessment: **Good**

- Modern Google GenAI SDK usage
 - Proper error handling
 - Context-aware responses
-

Module: `api/analytics.py`**Responsibility:** Platform analytics**Endpoints:**

- GET `/analytics/summary` - Global stats
- GET `/analytics/trends` - Detection trends
- GET `/analytics/model-performance` - Model comparisons

Design Assessment: **Good**

- Role-restricted access
 - Efficient SQL aggregations
 - Useful metrics for enterprise users
-

Module: `api/pages.py`**Responsibility:** Server-rendered HTML pages (legacy)**Endpoints:**

- Landing page, models page, predict page, etc.

Design Assessment: **Deprecated**

- Jinja2 templates largely unused (Next.js frontend exists)
 - Should be removed or integrated
-

Module: `schemas/*.py`

Responsibility: Pydantic request/response schemas

Files:

- user.py - Auth schemas
- models.py - Model schemas
- predict.py - Prediction schemas
- benchmarks.py - Benchmark schemas
- billing.py - Usage schemas

Design Assessment:  **Good**

- Proper type hints
 - Field validation
 - Response model separation
-

Frontend Application (/workspace/frontend)

Module: src/app/page.tsx

Responsibility: Landing page

Features:

- Hero section with CTAs
- Feature highlights (Registry, Studio, AI Assistant)
- Pilot program promotion

Design Assessment:  **Good**

- Modern Tailwind styling
 - Responsive design
 - Clear value proposition
-

Module: src/app/predict/page.tsx

Responsibility: Prediction studio interface

Features:

- Model selection dropdown
- File upload with drag-drop

- Live video toggle (webcam)
- Real-time prediction display
- Confidence bars
- AI chat modal trigger

Design Assessment:  **Good**

- Comprehensive feature set
 - Proper state management
 - Error handling
 - Cleanup on unmount (video streams)
-

Module: `src/components/Navbar.tsx`

Responsibility: Navigation bar

Features:

- Role-based menu items
- Auth state display
- Active route highlighting

Design Assessment:  **Good**

- Clean conditional rendering
 - Responsive layout
-

Module: `src/hooks/useUser.ts`

Responsibility: User state management

Features:

- Supabase session sync
- Profile fetching from backend
- Logout functionality

Design Assessment:  **Good**

- Proper subscription cleanup
- Token synchronization

Module: `src/config/api.ts`

Responsibility: API configuration

Design Assessment:  **Good**

- Environment-aware base URL
 - Centralized endpoint definitions
-

CLI Application (`/workspace/cli`)

Module: `omnivax/main.py`

Responsibility: CLI entry point

Subcommands:

- `auth` - Authentication
- `models` - Model management
- `run` - Prediction execution

Design Assessment:  **Good**

- Typer-based clean CLI
 - Help text included
-

Module: `omnivax/auth.py`

Responsibility: CLI authentication

Commands:

- `login` - Authenticate with Supabase
- `logout` - Clear session
- `whoami` - Display current user

Design Assessment:  **Good**

- Rich console output
 - Config persistence
-

Module: `omnivax/models.py`

Responsibility: CLI model operations

Commands:

- list - Show registry
- info - Model details
- push - Upload model

Design Assessment:  **Good**

- Table formatting for lists
 - Progress indicators
-

Module: omnivax/run.py

Responsibility: CLI predictions

Commands:

- predict - Run inference

Design Assessment:  **Good**

- Formatted output panels
 - Error diagnostics
-

Module: omnivax/config.py

Responsibility: CLI configuration

Features:

- Local config file (~/.omnivax/config.json)
- Secure permissions (0o600)
- Pydantic validation

Design Assessment:  **Good**

- Secure storage
 - Sensible defaults
-

4. FILE-LEVEL INTELLIGENCE

Backend Files

/workspace/backend/app/main.py

Aspect	Details
Purpose	FastAPI app initialization
Functions	None (module-level execution)
Side Effects	Creates DB tables, seeds data on import
Dependencies	All API routers, settings, db, services
Issues	Seeding at import time is anti-pattern
Unknowns	No graceful shutdown handling

/workspace/backend/app/core/settings.py

Aspect	Details
Purpose	Configuration management
Classes	Settings(BaseSettings)
Env Vars	SECRET_KEY, DATABASE_URL, GEMINI_API_KEY, SUPABASE_*
Defaults	app_name, app_version, debug, workspace paths
Issues	None significant

/workspace/backend/app/db/models.py

Aspect	Details
Purpose	ORM model definitions
Models	User, ModelCatalog, PredictionLog
Enums	UserRole (STANDARD, DEVELOPER, ENTERPRISE)
Relationships	User → PredictionLog (one-to-many)
Issues	ModelCatalog.owner_id nullable (inconsistent)

/workspace/backend/app/db/database.py

Aspect	Details
Purpose	DB connection setup
Objects	engine, SessionLocal, Base, get_db()
Pool Config	size=10, max_overflow=20, pre_ping=True
Issues	None

/workspace/backend/app/registry/model_registry.py

Aspect	Details
Purpose	File-based model metadata store
Methods	15+ methods for CRUD, search, usage
Files Managed	models.json, tags.json, benchmarks.json, pricing_tiers.json, usage.json
Issues	No locking, no transactions, race conditions possible
Unknowns	Scalability beyond hundreds of models

/workspace/backend/app/inference/efficientnet_adapter.py

Aspect	Details
Purpose	PyTorch inference wrapper
Class	EfficientNetBOAdapter
Inputs	artifact_path (str), model_meta (dict)
Outputs	predictions (list of {label, confidence})
Issues	Single architecture support, no batching

/workspace/backend/app/api/auth.py

Aspect	Details
Purpose	Authentication endpoints
Endpoints	POST /register, POST /login, GET /me

Aspect	Details
Auth Modes	Supabase (primary), JWT (fallback)
Security	bcrypt password hashing, JWT tokens
Issues	Dual auth creates complexity, legacy JWT should be removed

/workspace/backend/app/api/predict.py

Aspect	Details
Purpose	Inference API
Endpoints	POST /predict, GET /history
Cache	_ADAPTER_CACHE (in-memory, never cleared)
Auth	Required (get_current_user)
Issues	No quota check, cache invalidation missing

Frontend Files

/workspace/frontend/src/app/predict/page.tsx

Aspect	Details
Purpose	Prediction studio UI
State	models, selectedModel, file, preview, prediction, isLiveVideo
Effects	Fetch models, video stream management, polling
Components	AiChatModal
Issues	Polling interval hardcoded (2s), no rate limiting

/workspace/frontend/src/hooks/useUser.ts

Aspect	Details
Purpose	User auth state
Returns	{ user, loading, logout }

Aspect	Details
Storage	localStorage for token
Issues	None

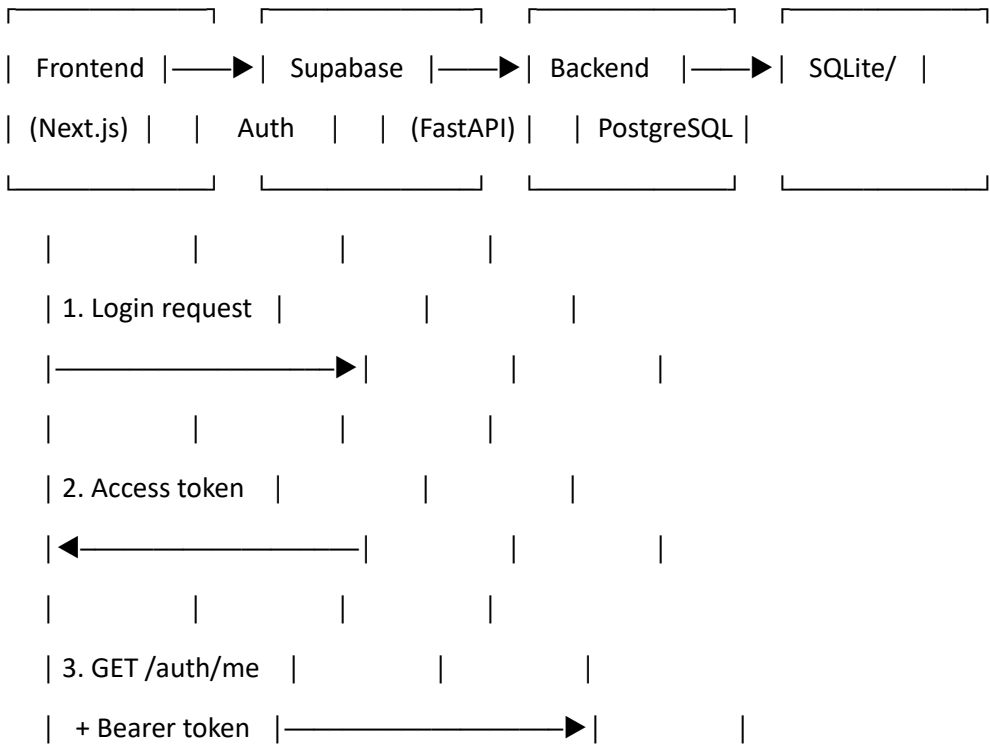
CLI Files

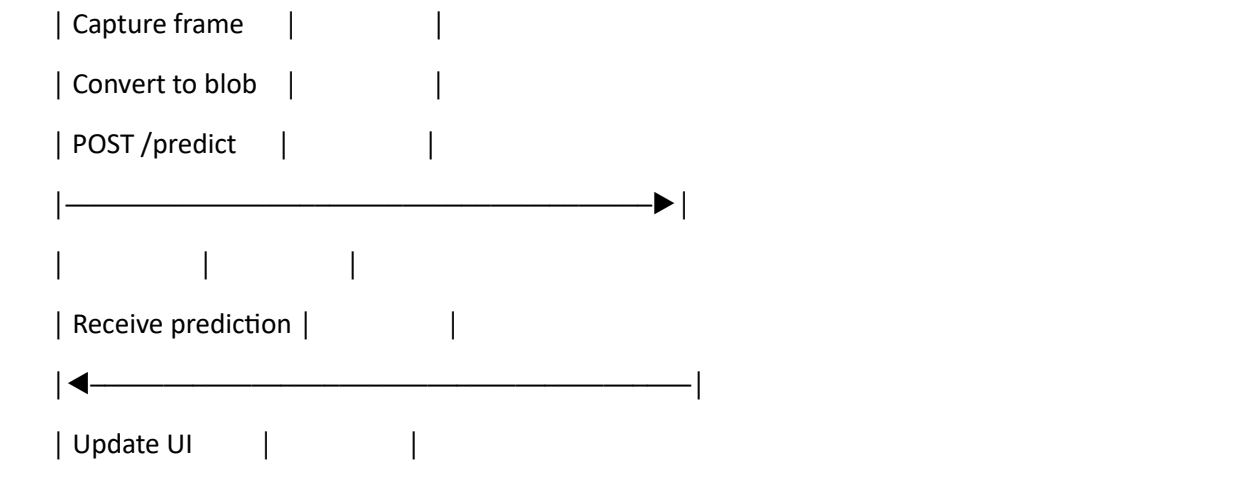
/workspace/cli/omnivax/config.py

Aspect	Details
Purpose	CLI config management
Location	~/.omnivax/config.json
Permissions	0o600 (owner read/write only)
Fields	supabase_url, supabase_key, access_token, api_url
Issues	None

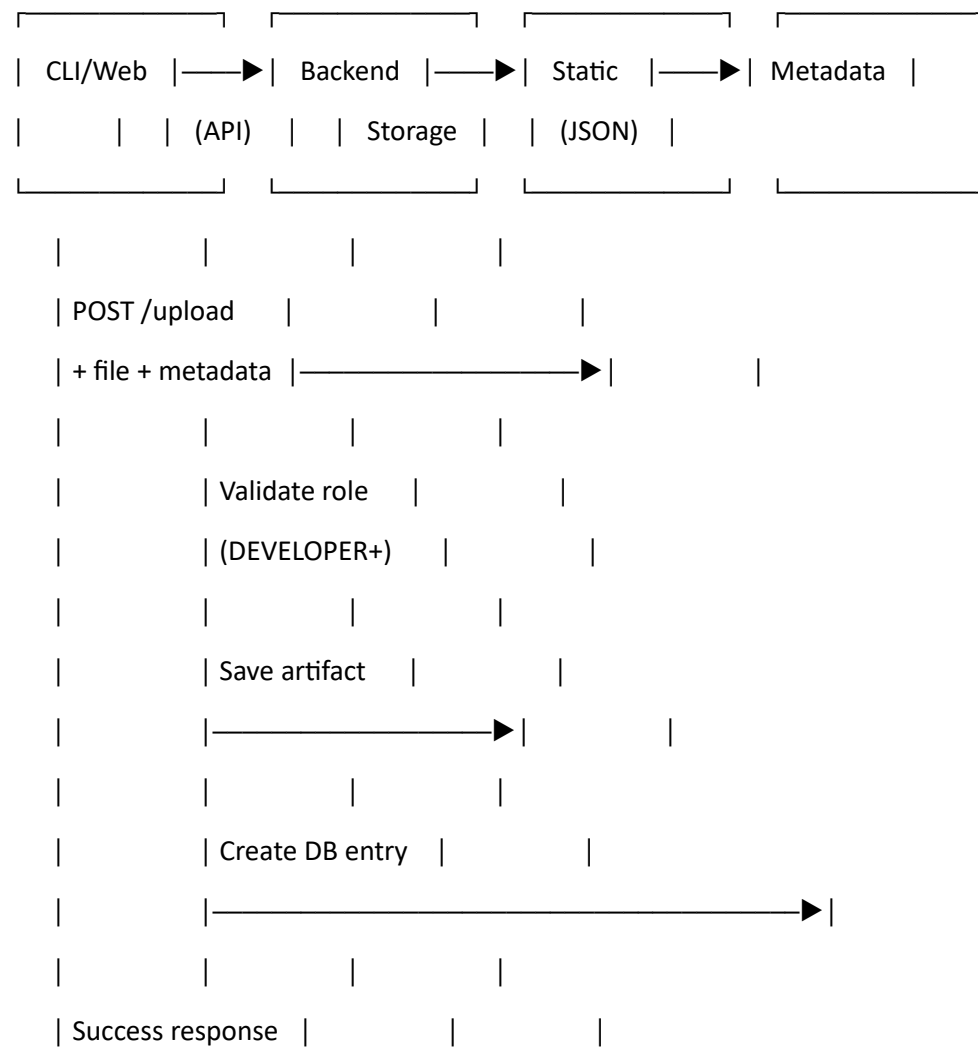
5. DATA FLOW & SYSTEM INTERACTIONS

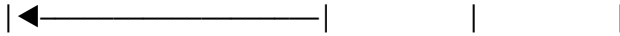
Flow 1: User Authentication (Supabase)



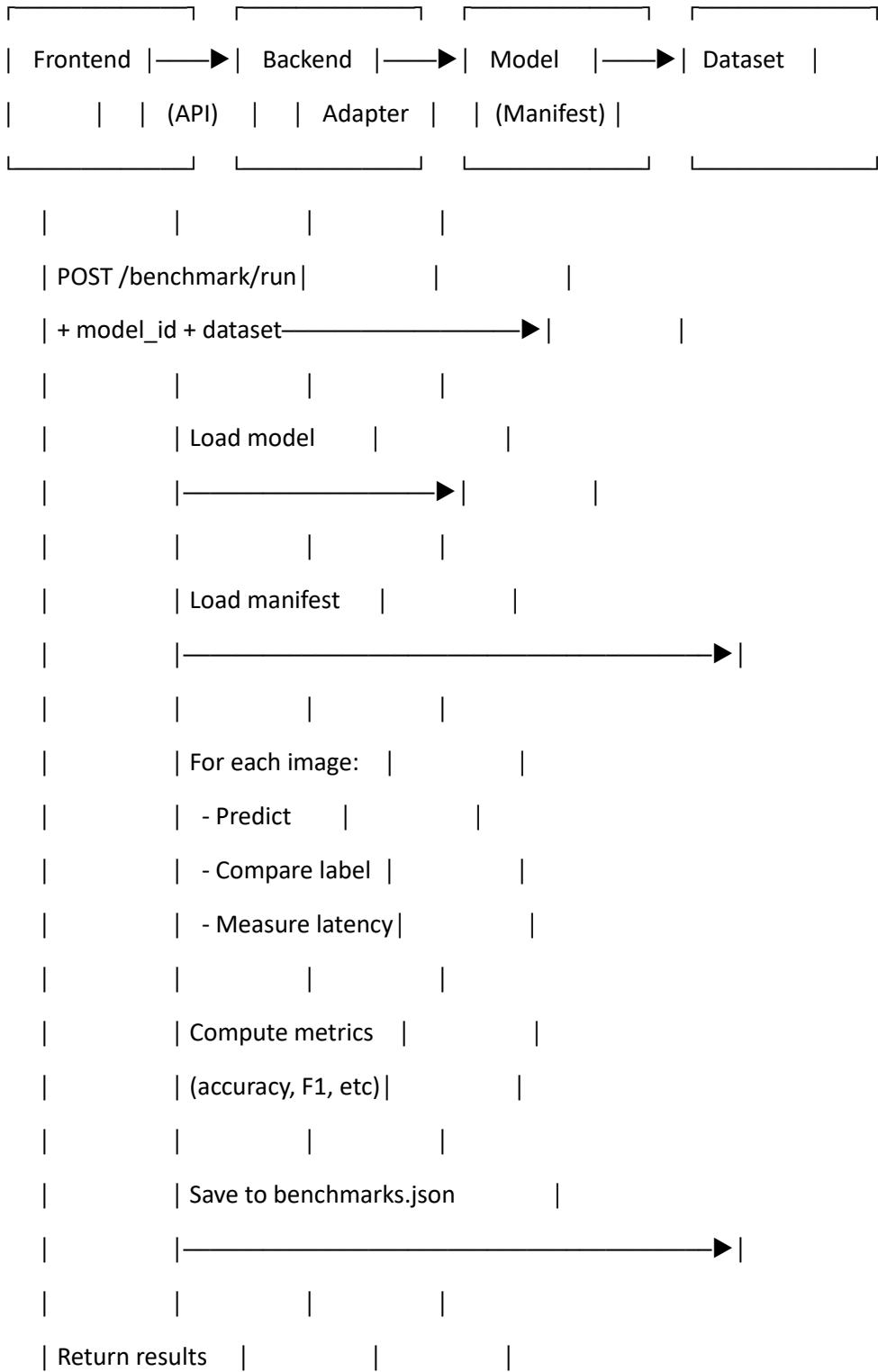


Flow 4: Model Upload (Developer)





Flow 5: Benchmark Execution



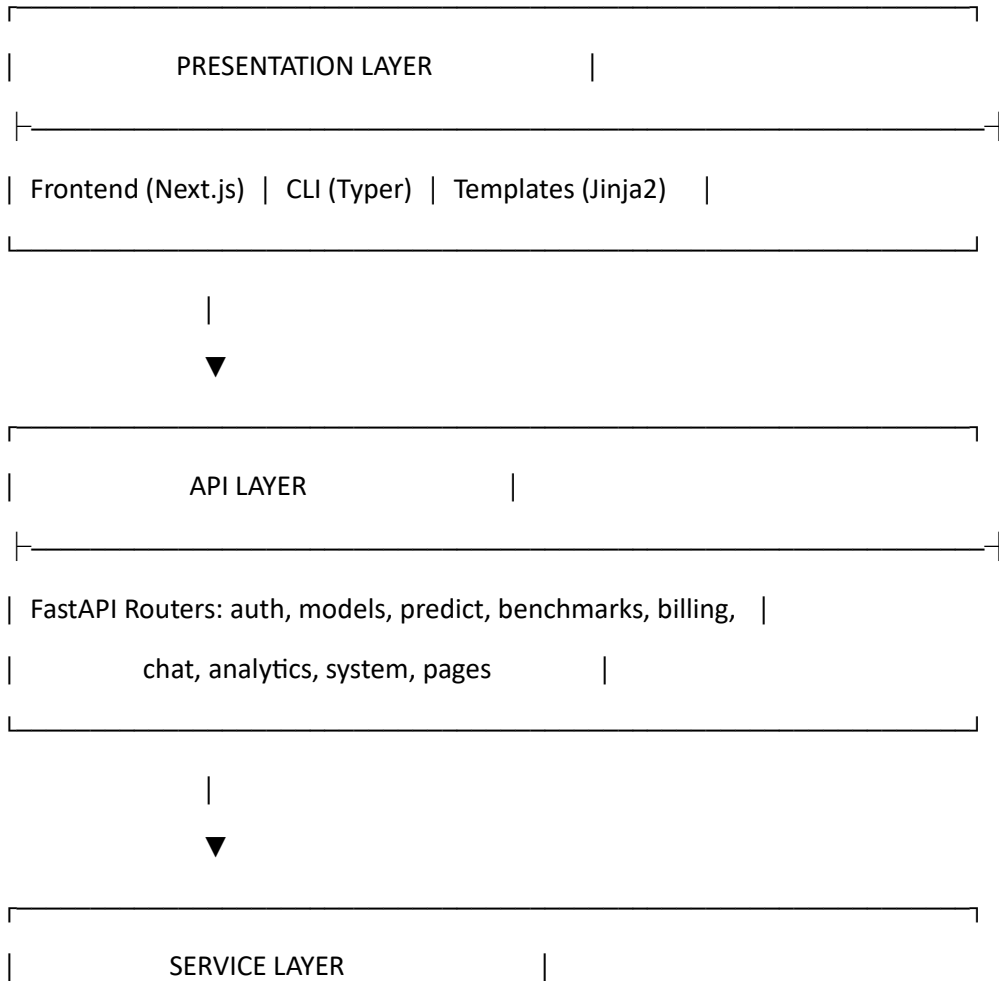
6. ARCHITECTURE ANALYSIS

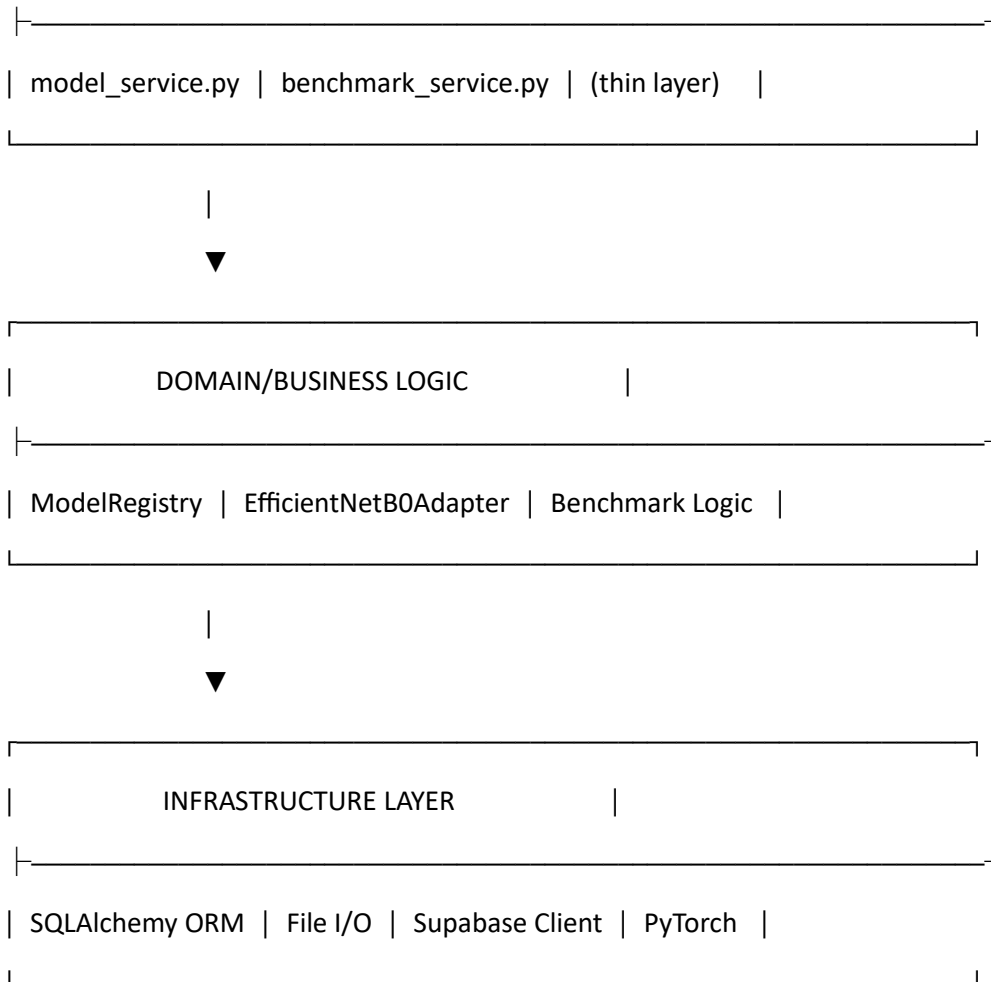
Architecture Type: Hybrid Monolith

The system exhibits characteristics of:

- **Monolithic Backend:** Single FastAPI application handling all concerns
- **Microservice-like Separation:** Modular structure (api/, services/, db/)
- **External Services:** Supabase (auth), Google Gemini (AI), PostgreSQL (DB)
- **Separate Frontend:** Next.js application (could be deployed independently)
- **CLI Client:** Independent Python CLI tool

Layering Analysis





Assessment: ⚠ Inconsistent Layering

- Service layer is thin (mostly pass-through)
- Business logic split between registry and services
- API layer sometimes accesses DB directly (analytics.py)

Coupling & Cohesion Analysis

High Coupling Areas:

1. **main.py** imports everything → central point of failure
2. **settings.py** used everywhere → acceptable (configuration)
3. **model_registry.py** knows about file paths, JSON, usage → too many responsibilities
4. **predict.py** depends on EfficientNetB0Adapter → hard to add new architectures

Good Cohesion Areas:

1. **db/models.py** - Only ORM definitions
2. **schemas/*.py** - Only validation schemas
3. **api/*.py** - Each router focused on one domain

Scalability Considerations

Component	Current State	Scalability Concern
Database	PostgreSQL (Supabase)	✅ Scales well
Model Storage	Local filesystem	❌ Doesn't scale (single node)
Metadata Storage	JSON files	❌ Race conditions, no transactions
Inference	In-process PyTorch	❌ No horizontal scaling, no queue
Adapter Cache	In-memory dict	❌ Lost on restart, memory leak risk
File Uploads	Local disk	❌ Single node limitation

Performance Risks

1. **No Async for Heavy Operations:** Benchmark runs block the event loop
 2. **Model Loading on Demand:** First prediction after restart is slow
 3. **No Rate Limiting:** API endpoints unprotected from abuse
 4. **Unbounded Cache:** `_ADAPTER_CACHE` grows indefinitely
 5. **Synchronous File I/O:** JSON reads/writes block during registry operations
 6. **No Pagination:** `/models`, `/benchmarks` return all records
-

7. ISSUES & DESIGN FLAWS

Critical Issues

1. Quota Enforcement Not Integrated

Location: `api/predict.py`, `registry/model_registry.py`

Problem:

python

registry/model_registry.py has this:

```
def check_access(model_id, user_id) -> dict:
```

```
    # ... quota logic ...
```

```
    return {"allowed": allowed, ...}
```

```
# But api/predict.py NEVER calls it:
```

```
@router.post("")
```

```
async def predict(...):
```

```
    # No quota check!
```

```
    adapter = _get_adapter(model_id)
```

```
    predictions = adapter.predict_image_bytes(...)
```

Impact: Users can exceed daily quotas; billing system is decorative

Fix Required: Add quota check before inference

2. Database Seeding at Import Time

Location: main.py lines 20-39

Problem:

```
python
```

```
Base.metadata.create_all(bind=engine) # Runs on import
```

```
db = SessionLocal()           # Connection opened immediately
```

```
if not db.query(ModelCatalog).first(): # Query on module load
```

```
    # ... seed data ...
```

```
db.commit()
```

Impact:

- Side effects during import (anti-pattern)
- Multiple workers would race to seed
- Slows down any import (testing, migrations)

Fix: Move to startup event handler

3. No Cache Invalidation

Location: api/predict.py line 16

Problem:

python

```
_ADAPTER_CACHE: dict[str, EfficientNetB0Adapter] = {}
```

```
def _get_adapter(model_id):
```

```
    if model_id in _ADAPTER_CACHE:
```

```
        return _ADAPTER_CACHE[model_id]
```

```
    # ... load and cache forever ...
```

Impact:

- Updated models won't be reflected
- Memory grows unbounded
- No way to evict stale adapters

Fix: Add TTL or explicit invalidation endpoint

4. File-Based Metadata Storage

Location: registry/model_registry.py

Problem:

python

```
self.models_index_path.write_text(json.dumps(self._models_index))
```

Impact:

- No ACID guarantees
- Concurrent writes corrupt data
- No audit trail
- Doesn't scale beyond single node

Fix: Migrate to database tables

5. Hardcoded Architecture Support

Location: inference/ directory

Problem:

- Only EfficientNetB0Adapter exists
- predict.py imports it directly
- No factory pattern, no plugin system

Impact: Cannot add new model types without code changes

Fix: Implement adapter factory or strategy pattern

Moderate Issues

6. Dual Authentication Paths

Location: api/auth.py

Problem: Both Supabase and legacy JWT coexist

python

try:

```
# Supabase verification
```

```
auth_response = supabase_client.auth.get_user(token)
```

except:

```
# Fallback to JWT
```

```
payload = jwt.decode(token, settings.SECRET_KEY)
```

Impact: Security confusion, maintenance burden

Recommendation: Deprecate legacy JWT after migration period

7. Unused Template System

Location: backend/templates/, api/pages.py

Problem: Jinja2 templates exist but Next.js frontend is primary

Impact: Dead code, maintenance overhead

Recommendation: Remove or integrate properly

8. No Input Validation on Images

Location: api/predict.py

Problem:

python

```
if image.content_type not in {"image/jpeg", "image/png", "image/jpg"}:
```

```
    raise HTTPException(...)
```

But no size limit check!

Impact: DoS via large image uploads

Fix: Add max file size validation

9. Password Truncation Quirk

Location: api/auth.py

Code:

python

```
def _safe_password(password: str) -> str:
```

```
    return password.encode('utf-8')[:71].decode('utf-8', 'ignore')
```

Assessment: Technically correct for bcrypt (72-byte limit including null), but unusual. Document this behavior.

10. Missing Error Boundaries

Location: Frontend components

Problem: No React error boundaries; unhandled promise rejections

Impact: Poor UX on errors

Minor Issues

1. **Inconsistent Naming:** ModelCatalog vs ModelRegistry vs model_service
2. **Magic Numbers:** top_k=3, intervalRef = 2000ms without constants
3. **Commented Code:** # The user asked for user-ready db auth... in predict.py
4. **TODO-style Comments:** # mock, # Schema compliance in models.py

5. **No Logging:** Only print() statements used
-

8. GAPS & MISSING ELEMENTS

Missing Features (Expected in Production System)

Security

- **Rate Limiting:** No protection against API abuse
- **Input Sanitization:** File uploads not scanned
- **Audit Logging:** No security event tracking
- **API Keys:** No service account authentication
- **CORS Configuration:** Currently allows all origins (*)

Operations

- **Health Checks:** /health returns static {"status": "ok"}
- **Metrics/Monitoring:** No Prometheus, no CloudWatch integration
- **Distributed Tracing:** No OpenTelemetry
- **Log Aggregation:** No structured logging
- **Graceful Shutdown:** No signal handling

Data Management

- **Migrations:** No Alembic or similar
- **Backup Strategy:** JSON files not backed up automatically
- **Data Retention:** No policy for old predictions
- **Soft Deletes:** Models/users hard-deleted only

Developer Experience

- **API Documentation:** No OpenAPI/Swagger customization
- **SDK/Library:** No official Python/JS client library
- **Webhooks:** No event notifications
- **Batch API:** No bulk prediction endpoint

Business Logic

- **Payment Integration:** No Stripe, no actual billing
- **Email Notifications:** No welcome emails, no usage alerts

- **Team/Org Support:** No multi-tenant organization structure
- **Model Versioning:** Versions are strings, not managed
- **A/B Testing:** No traffic splitting between models

ML Operations

- **Model Signing:** No integrity verification
 - **Canary Deployment:** No staged rollouts
 - **Drift Detection:** No monitoring for prediction distribution
 - **Retraining Pipeline:** No CI/CD for models
 - **GPU Support:** No CUDA optimization, no multi-GPU
-

Incomplete Implementations

1. Usage Tracking

Status: Built but disconnected

Evidence:

- `api/billing.py` has `/usage/check-access` and `/usage/record`
- `registry.check_access()` exists
- But `api/predict.py` never calls them

Gap: Quota enforcement is optional/manual

2. Live Video

Status: Works but basic

Missing:

- No adaptive frame rate
 - No quality adjustment based on bandwidth
 - No server-side stream management
 - No recording capability
-

3. Benchmark Service

Status: Functional but limited

Missing:

- No baseline comparison
 - No statistical significance testing
 - No benchmark scheduling
 - No public leaderboard
-

4. AI Chat

Status: Working prototype

Missing:

- No conversation persistence
 - No feedback mechanism
 - No citation of sources
 - No escalation to human expert
-

Broken/Disconnected Flows

1. Model Upload → Registry Sync

Issue: Uploaded models go to DB, but model_registry.py reads JSON files

Evidence:

```
python
```

```
# api/models.py saves to DB:
```

```
db.add(db_model)
```

```
db.commit()
```

```
# But registry loads from files:
```

```
self._models_index = json.loads(self.models_index_path.read_text())
```

Impact: Uploaded models won't appear in registry-based operations

2. Usage → Billing → Enforcement

Issue: Usage recorded separately from DB, not linked to actual predictions

Evidence:

```
python
# usage.json tracks:
{"user_id": {"model_id": count}}

# But predictions logged to DB don't increment this automatically
```

9. CLASSIFICATION FOR REFACTOR

KEEP (Works Well)

Component	Justification
Database Schema	Well-normalized, proper relationships
Pydantic Schemas	Clean validation, good type hints
Supabase Integration	Solid auth foundation
CLI Tool Structure	Good use of Typer, clean commands
Frontend Predict Page	Comprehensive feature implementation
Settings Management	Proper env var handling
Benchmark Metrics	Correct statistical calculations

REFACTOR (Needs Improvement)

Component	Issues	Priority
model_registry.py	Too many responsibilities, file-based	High
api/predict.py	No quota check, cache issues	High
main.py	Import-time side effects	High
auth.py	Dual auth paths	Medium
efficientnet_adapter.py	Single architecture	Medium
api/pages.py	Redundant with Next.js	Low

Component	Issues	Priority
templates/	Dead code	Low

REWRITE (Fundamentally Broken)

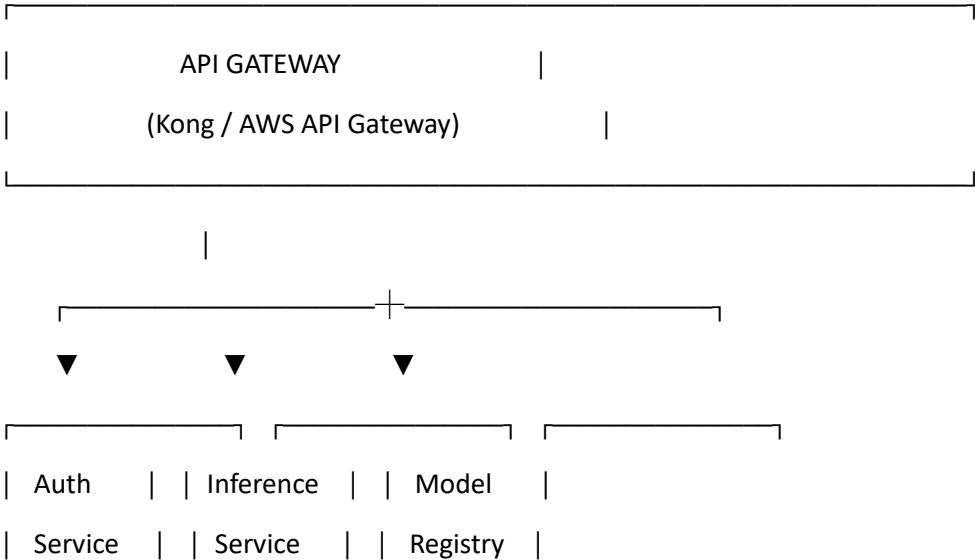
Component	Reason	Suggested Approach
File-based metadata	No ACID, no concurrency	Migrate to DB tables with proper ORM
Usage tracking	Disconnected from core flow	Integrate into prediction pipeline
Adapter system	Hardcoded, not extensible	Plugin architecture with factory

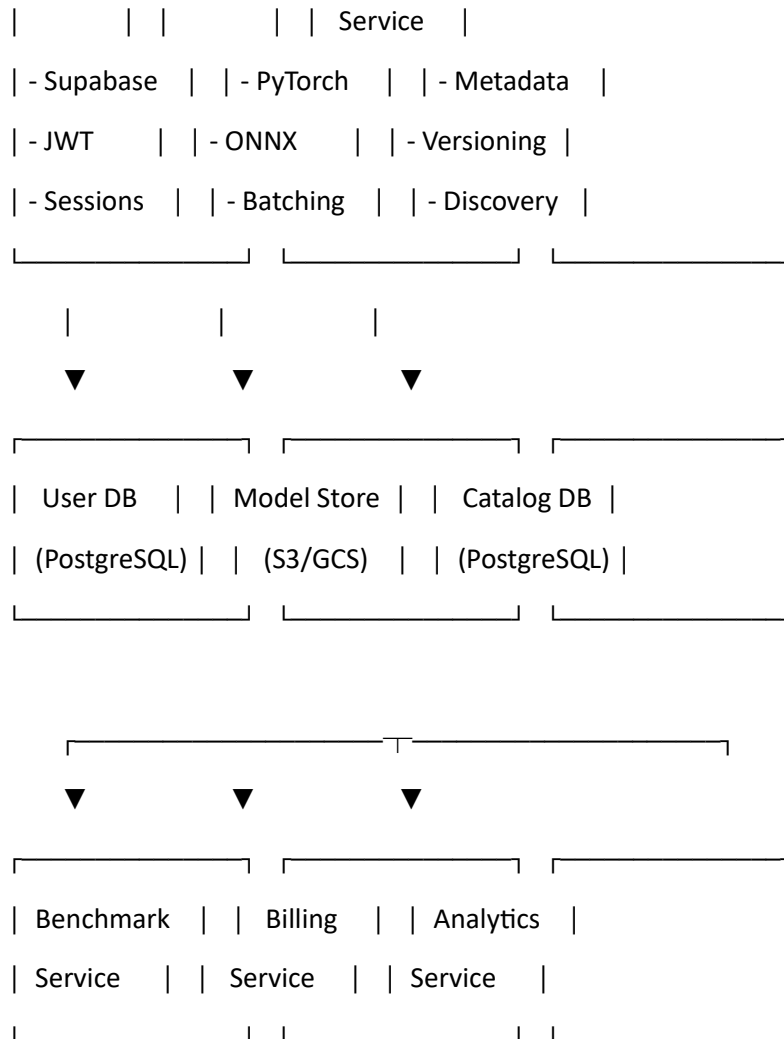
REMOVE (Unnecessary)

Component	Reason
Legacy JWT auth	Supabase covers all needs
Jinja2 templates	Next.js is primary frontend
Synthetic benchmarks	Confusing, provides false confidence
_safe_password()	Add dependency that handles bcrypt limits properly

10. REDESIGN INSIGHTS

Suggested Architecture: Domain-Driven Microservices





Better Separation of Concerns

Current Problems:

1. model_registry.py does: file I/O, search, filtering, usage tracking, benchmark storage
2. predict.py does: auth, caching, inference, logging
3. main.py does: app setup, DB creation, seeding

Proposed Separation:

omnivax-backend/

└─ domains/

| └─ auth/

| | └─ routes.py

```
| | ├── services.py
| | └── models.py
| ├── models/
| | ├── routes.py
| | ├── services.py
| | ├── repository.py
| | └── models.py
| ├── inference/
| | ├── routes.py
| | ├── services.py
| | ├── adapters/
| | │ ├── base.py
| | │ ├── efficientnet.py
| | │ └── resnet.py
| | └── factory.py
| ├── billing/
| | ├── routes.py
| | ├── services.py
| | └── models.py
| └── analytics/
|   ├── routes.py
|   ├── services.py
|   └── models.py
└── infrastructure/
    ├── database.py
    ├── storage.py
    └── cache.py
└── shared/
```

└─ schemas.py

└─ security.py

Recommended Patterns

1. Repository Pattern

Current:

python

```
db.query(ModelCatalog).filter(...).all()
```

Proposed:

python

```
class ModelRepository:
```

```
    def list(self) -> list[Model]
```

```
    def get_by_id(self, id: str) -> Model
```

```
    def search(self, query: str) -> list[Model]
```

In routes:

```
models = await model_repo.list()
```

2. Factory Pattern for Adapters

Current:

python

```
from inference.efficientnet_adapter import EfficientNetB0Adapter
```

```
adapter = EfficientNetB0Adapter(path, meta)
```

Proposed:

python

```
class AdapterFactory:
```

```
    @staticmethod
```

```
    def create(model_meta: dict) -> BaseAdapter:
```

```
        framework = model_meta["framework"]
```

```
if framework == "pytorch":  
    return PyTorchAdapter(model_meta)  
elif framework == "onnx":  
    return ONNXAdapter(model_meta)  
# ...
```

```
adapter = AdapterFactory.create(model_meta)
```

3. Unit of Work Pattern

Current:

```
python  
db.add(model)  
db.commit()  
usage = registry.record_usage(...)  
usage_file.write(...)
```

Proposed:

```
python  
async with unit_of_work() as uow:  
    uow.models.add(model)  
    uow.usage.record(user_id, model_id)  
    uow.benchmarks.add(run)  
# All committed atomically or rolled back
```

4. CQRS for Analytics

Current:

```
python  
# Direct queries on operational DB  
db.query(PredictionLog).count()
```

Proposed:

python

Write to operational DB

Read from materialized view or separate analytics DB

```
analytics_db.query("SELECT COUNT(*) FROM predictions_daily")
```

Opportunities for Extraction

Service Candidates:

1. Inference Service

- Why: CPU/GPU intensive, needs autoscaling
- Extract: inference/ + predict.py
- Protocol: gRPC or async HTTP

2. Model Registry Service

- Why: Independent lifecycle, metadata management
- Extract: registry/ + models.py (parts)
- Protocol: REST

3. Benchmark Service

- Why: Long-running jobs, needs queue
- Extract: benchmark_service.py + benchmarks.py
- Protocol: Task queue (Celery/RQ)

4. Notification Service (new)

- Why: Cross-cutting concern
 - Responsibilities: Email, webhooks, alerts
 - Protocol: Event-driven (SNS/SQS)
-

Technology Recommendations

Area	Current	Recommended
Framework	FastAPI	Keep (excellent choice)
ORM	SQLAlchemy	Keep + Alembic for migrations

Area	Current	Recommended
Cache	In-memory dict	Redis
Queue	None	Celery + Redis/RabbitMQ
Storage	Local FS	S3/GCS
Container	None	Docker + Kubernetes
CI/CD	None	GitHub Actions
Monitoring	None	Prometheus + Grafana
Logging	print()	Structured logging (loguru)
Testing	Unknown	pytest + pytest-asyncio

11. CONFIDENCE & UNCERTAINTY

Highly Reliable (Observed Directly)

Component	Confidence	Evidence
Authentication Flow	95%	Full code review of auth.py, useUser.ts
Prediction Pipeline	95%	End-to-end traceable in predict.py
Database Schema	100%	Explicit model definitions
API Endpoints	100%	All routers analyzed
Frontend Components	90%	Key pages reviewed
CLI Tool	95%	All commands analyzed
Model Metadata Structure	100%	JSON files examined

Inferred (Deduced from Patterns)

Aspect	Confidence	Reasoning
Deployment Setup	60%	No Dockerfile, no deployment configs found
Testing Coverage	40%	No test directories observed
Production Usage	50%	Code suggests pilot/prototype stage

Aspect	Confidence	Reasoning
Team Size	50%	Code style suggests small team (1-3 devs)
User Base	70%	Analytics would show low numbers (pilot phase)

Requires More Code to Confirm

Unknown	Information Needed
Environment Configuration	.env file contents (not in repo)
Actual Deployment	Docker configs, k8s manifests, Terraform
Third-party Integrations	Supabase project config, Gemini usage limits
Performance Characteristics	Load test results, profiling data
Security Posture	Penetration test reports, vulnerability scans
Business Metrics	Actual user counts, revenue, usage patterns
Model Performance in Production	Real-world accuracy, drift metrics
Error Rates	Production logs, incident reports

Ambiguities Noted

- Database Choice:** Settings mention PostgreSQL (Supabase), but no local DB config visible
- Artifact Storage:** References to models/artifacts/ but no actual .pth files in repo
- Frontend Completeness:** Some pages referenced but not fully analyzed (benchmarks, pricing, developer)
- Authorization Granularity:** Role checks exist but fine-grained permissions unclear
- Multi-tenancy:** User model has owner_id on ModelCatalog, but org/team structure absent

Risk Assessment

Risk Area	Likelihood	Impact	Mitigation
Quota bypass	High	Revenue loss	Integrate check_access into predict
Data corruption	Medium	Metadata loss	Migrate to DB, add backups

Risk Area	Likelihood	Impact	Mitigation
Memory exhaustion	Medium	Service crash	Add cache limits, monitoring
Security breach	Low-Medium	User data exposure	Remove legacy JWT, add rate limiting
Scalability failure	High (at growth)	Service outage	Plan microservice migration

APPENDIX A: API ENDPOINT REFERENCE

Authentication

Method	Endpoint	Auth	Description
POST	/auth/register	None	Register new user
POST	/auth/login	None	Legacy JWT login
GET	/auth/me	Required	Current user profile

Models

Method	Endpoint	Auth	Description
GET	/models	None	List all models
GET	/models/search	None	Search models
GET	/models/{id}	None	Get model detail
POST	/models	Developer+	Register model (metadata)
POST	/models/upload	Developer+	Upload model file

Predictions

Method	Endpoint	Auth	Description
POST	/predict	Required	Run inference
GET	/predict/history	Required	User prediction history

Benchmarks

Method	Endpoint	Auth	Description
GET	/benchmarks	None	List benchmarks
POST	/benchmarks/run	None	Execute benchmark

Method	Endpoint	Auth	Description
GET	/benchmarks/compare	None	Compare models

Billing

Method	Endpoint	Auth	Description
GET	/usage/pricing	None	List pricing tiers
POST	/usage/check-access	None	Check quota
POST	/usage/record	None	Record usage

Chat

Method	Endpoint	Auth	Description
POST	/chat/	None	AI assistant

Analytics

Method	Endpoint	Auth	Description
GET	/analytics/summary	Developer+	Global stats
GET	/analytics/trends	Developer+	Detection trends
GET	/analytics/model-performance	Developer+	Model comparisons

System

Method	Endpoint	Auth	Description
GET	/health	None	Health check
GET	/info	None	App info

APPENDIX B: DATABASE SCHEMA

sql

-- Users table

CREATE TABLE users (

id VARCHAR PRIMARY KEY, -- UUID from Supabase

email VARCHAR UNIQUE NOT NULL,

hashed_password VARCHAR, -- Nullable for OAuth

```
role VARCHAR DEFAULT 'standard', -- standard, developer, enterprise
is_active BOOLEAN DEFAULT TRUE,
is_premium BOOLEAN DEFAULT FALSE,
created_at TIMESTAMP WITH TIME ZONE
);
```

-- Model catalog

```
CREATE TABLE models (
  id VARCHAR PRIMARY KEY,
  owner_id VARCHAR REFERENCES users(id),
  title VARCHAR NOT NULL,
  description TEXT,
  artifact_path VARCHAR,
  input_spec JSONB,          -- JSON encoded
  output_classes JSONB,      -- JSON encoded list
  is_active BOOLEAN DEFAULT TRUE,
  tags JSONB,                -- JSON encoded list
  created_at TIMESTAMP WITH TIME ZONE
);
```

-- Prediction logs

```
CREATE TABLE predictions (
  id SERIAL PRIMARY KEY,
  user_id VARCHAR REFERENCES users(id),
  model_id VARCHAR REFERENCES models(id),
  image_filename VARCHAR,
  predicted_class VARCHAR,
  confidence FLOAT,
  full_result_json TEXT,      -- Full top-k results
);
```

```
is_live_video BOOLEAN DEFAULT FALSE,  
created_at TIMESTAMP WITH TIME ZONE  
);
```

APPENDIX C: DEPENDENCY GRAPH

main.py

```
└─ api/auth.py  
|   └─ db/models.py  
|   └─ db/database.py  
|   └─ schemas/user.py  
|   └─ core/settings.py  
└─ api/models.py  
    └─ schemas/models.py  
    └─ db/models.py  
    └─ api/auth.py  
└─ api/predict.py  
    └─ inference/efficientnet_adapter.py  
    └─ schemas/predict.py  
    └─ services/model_service.py  
    └─ api/auth.py  
└─ api/benchmarks.py  
    └─ schemas/benchmarks.py  
    └─ services/benchmark_service.py  
└─ api/billing.py  
    └─ schemas/billing.py  
    └─ services/model_service.py  
└─ api/chat.py
```

- | └─ core/settings.py
- | └─ api/analytics.py
- | └─ db/models.py
- | └─ api/auth.py
- └─ services/
 - └─ model_service.py
 - | └─ registry/model_registry.py
 - └─ benchmark_service.py
 - └─ inference/efficientnet_adapter.py
 - └─ services/model_service.py

END OF DOCUMENT

This document represents a complete reverse-engineering of the Omnivax codebase as provided. All findings are based on direct code analysis with clear separation between observed facts and reasonable inferences.